

SIGNALS.md

Exported: June 07, 2026

Summary

This document outlines a sophisticated **algorithmic framework** designed to convert complex market data into binary trading decisions for options. By analyzing the relationship between **implied volatility, dealer gamma positioning, and institutional volume**, the system identifies high-probability setups while actively filtering out “traps” where time decay would normally erode profits. The core logic relies on a **weighted execution matrix** that balances multiple independent sub-signals against a momentum index to ensure every trade has a mathematical edge. Finally, the strategy applies a **time-of-day multiplier** to adjust its sensitivity, ensuring the system remains disciplined during volatile market openings and cautious during quiet midday periods.

Keywords: Signal Logic pipeline, Option Greek metrics, Market regime analysis, Master Execution Matrix, Trading confidence scores

Content

Signal Logic — Master Execution Matrix This document describes the composite signal pipeline that maps multi-feature state into a discrete GO / NO-GO / HOLD decision.

Inputs (Computed Per Tick) Sub-Signals (4 Independent Checks)

1. Vol-OI Nexus — **the bull-trap detector** Why it works: High PCR (>1.1) often means put-heavy sentiment. If IV is **crushing** simultaneously, the puts were written into a vol spike that’s now fading — buying calls on a “put wall bounce” then bleeds premium.
2. Gamma Hedging Trigger — **the dealer squeeze detector** Why it works: When dealers are **net short gamma**, a move in either direction forces them to buy the underlying in the direction of the move, amplifying it. Crossing a major OI wall under negative GEX is the textbook explosive setup.
3. Premium-Spot Lead-Lag — **institutional accumulation** Why it works: When spot is range-bound but call premiums are **rising above** their VWAP with PCR climbing, it means options flow is leading the underlying. This is institutional accumulation by definition (you can’t make spot move without an OI footprint, but you **can** quietly accumulate calls).
- 4.

No-Trade Trap Signature — **the strangle-write filter** Why it works: Balanced high OI on both sides + flat PCR + stationary Max Pain = someone sold a straddle/strangle. Time decay will collect the premium from any small move. Buying options here is paying for nothing.

Momentum Index (Continuous Score in $[-1, 1]$) Default weights from config/settings.yaml: $w_{\text{pcr_slope}} = 0.30$ $w_{\text{vwap_gap}} = 0.40$ (heaviest — direct flow signal) $w_{\text{delta_flow}} = 0.30$ (GEX-derived) Threshold: $> 0.55 \rightarrow$ buy bias; $< -0.55 \rightarrow$ exit/no-trade bias.

Master Execution Matrix After the 4 sub-signals + momentum index are computed, the engine evaluates a weighted vote against the time-of-day multiplier: The decision is: Time-of-Day Multipliers Confidence Score $\text{confidence} = \min(1.0, \text{abs_score} / 6.0)$ $< 0.3 \rightarrow$ weak; dashboard “HOLD” colored grey $0.3-0.6 \rightarrow$ moderate; sub-signal partial match $> 0.6 \rightarrow$ strong; ready to trade (subject to order manager risk)

Suggested Strike For GO + CE: returns call_wall (the strike with the largest positive gamma nearby) For GO + PE: returns put_wall For NO-GO: returns the wall anyway (for monitoring)

In practice, the order manager applies further filtering (max notional, position size, cooldown) before submitting.

Worked Example Setup at 11:45 IST, NIFTY spot = 24,000 $\rightarrow$ Sub-signal vol_oi: NO_TRADE (high PCR + IV crushing = bull trap) $\rightarrow$ Sub-signal gamma: HOLD (positive GEX, not near wall) $\rightarrow$ Sub-signal lead-lag: HOLD (VWAP gap too small) $\rightarrow$ Sub-signal no_trade: NO_TRADE (strangle flag + flat PCR + static max pain) $\rightarrow$ Momentum index: +0.08 (neutral) $\rightarrow$ Time block: midday $\rightarrow$ multiplier 1.4 $\rightarrow$ required_score = 4 $\rightarrow$ go_score = 0, no_go_score = 2 $\rightarrow$ NO-GO with high confidence Action: Stay out. If entered on a long, exit on next bar.

Tuning: All thresholds live in config/settings.yaml. After any change, run `python -m main backtest` and compare metrics.